Cloud Computing Mini Project: Distilled Command Parser on Kubernetes

Roll 230101018 | Repository: https://github.com/motheabhishek/cloud-project | Model release: v1.0

1. Overview

A Phi-3.5 teacher (served with Ollama) labels natural-language commands, and a Pythia-70M student is trained on the teacher's outputs (knowledge distillation). The student (v1) is served by a small FastAPI service (/healthz, /readyz, /parse) in a single-node k3d Kubernetes cluster, with SeaweedFS as object storage. Model weights are published as GitHub Release v1.0 and are not stored in Git. The whole system is rebuilt with **make up** from a fresh clone.

2. Data and cleaning

3000 teacher-labelled examples were cleaned with worker/clean.py. Rejected: 4 bad JSON, 7 bad keys, 0 bad intent, 81 bad slot and 48 duplicates (140 in total), leaving 2860 examples for training.

3. Model quality

Scores are percentages on 100 validation examples (results/score_*.txt).

Model	JSON valid	Intent	Date	Time	Place	Full frame
Untrained Pythia	0	0	0	0	0	0
Teacher (Phi-3.5)	82	59	65	76	71	39
Student v1 (deployed)	99	71	66	90	67	29
Student v2	97	65	74	87	77	34

The untrained model produced text for all 100 examples, but in the examples inspected it was not valid JSON in the required schema (repeated lists and unrelated key-value structures, cut off at the token limit), so every metric is 0. This is the baseline before distillation. After training, the student reaches 99% valid JSON and beats the teacher on JSON validity, intent and time. The teacher keeps the best full-frame score (39 against 29 for v1), because full frame needs every field correct at once. With 100 examples one point equals one example, so differences of 1 to 3 points are within noise. Student v2 is a second training run, better on date, place and full frame. v1 was deployed.

A repetition penalty of 1.1 gave all-zero scores for both students. In the examples inspected the output started correctly and then degraded into broken JSON, so default decoding was kept.

4. Deployment and load test

Standard Locust run: 4 users, 2 minutes, 2 replicas (measured on the first cluster, which was later rebuilt with make up and re-verified).

Measure	Value
Requests / failures	336 / 3 (0.89%)
p50 / p95	1200 ms / 2800 ms
Throughput	2.81 req/s
CPU per Pod	1943m and 1934m (limit 2000m)
Memory per Pod	731 MiB and 769 MiB (limit 1 GiB)
Replica split	165 / 168 (49.5% / 50.5%)

Measure	Value
Cold start	median 7 s (runs: 11 s, 7 s, 6 s)

5. Recovery and autoscaling

Recovery: with one Pod deleted under load, 456 requests gave 5 failures (1.10%), all HTTP 500, and the replacement Pod was Ready in about 18 s. HPA (target CPU 70%, 1 to 2 replicas): scaled up from 1 to 2 replicas at CPU 199-200%, and scaled back to 1 about 5 minutes after load ended (CPU fell to 0% at about 18 minutes of HPA age, replicas dropped at about 23 minutes; AGE has minute resolution). A second full run during the HPA test gave 843 requests, 9 failures (1.07%), p50 1500 ms, p95 3100 ms, 2.35 req/s.

6. Cost and SLO

Cost per replica-hour is 2 x 3.50 + 1 x 0.45 = 7.45 INR, so 2 replicas cost 14.90 INR/h. At 2.81 req/s (10116 req/h) this is 1.47 INR per 1000 requests. SLO: 99% of requests succeed per run. It was met in the standard run (99.11%) and narrowly missed in the recovery run (98.90%) and the HPA run (98.93%). The failures were all HTTP 500, consistent with the model's invalid-output rate (about 1%); this was not proven from the data.

7. Component analysis

Component	Limitation	Improvement
Data and storage	Single-node object storage, no replication; model downloaded at each start	Replicated storage, model cached on a volume
Model quality	70M student makes format and slot errors	More distillation data, constrained decoding
Latency and throughput	CPU-bound; 1 CPU gave p95 7700 ms before the fix	Quantised model, batching
Resource use	Pods saturated at 4 users	Smaller model, more replicas on multi-node cluster
Reliability	One node, no real fault tolerance; HPA at its maximum of 2 (about 190% vs 70% target)	Multi-node cluster, higher HPA maximum
Security and cost	Secret only base64; no TLS on Ingress	Sealed secrets, TLS, right-size CPU limit

8. Use of AI

Claude (Anthropic) was used in chat for planning, shell and kubectl commands, small Python patches, diagnosing problems (for example slow inference caused by PyTorch thread oversubscription under a 1-CPU limit), and drafting the NACL rules, subnet table and cost/SLO arithmetic. Every command was run and its output read before use. The NACL was checked with tools/nacl_sim.py (6/6 correct), model quality with tools/score.py, and timings with curl and Locust. AI mistakes (placeholder paths, a wrong CI job estimate, a Makefile default that ran training) were caught and fixed, and AI-suggested numbers were used only after checking them against files in results/. Teacher labels came from Phi-3.5 via Ollama and were not edited by hand. Details are in AI_USE.md.